

ASSIGNMENT 2

CO3



NAME:

V.MAHALAKSHMI



REG NO:

192521181



COURSE CODE:

CSA5113



COURSE:

CRYPTOGRAPHY AND
NETWORK SECURITY

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.

Answer:

MD5 is an outdated cryptographic hash function that produces a 128-bit hash value. It is not suitable for securely storing passwords because it is extremely fast, allowing attackers to perform a very large number of password guesses in a short time. MD5 also has known collision vulnerabilities and provides inadequate protection against modern password-cracking techniques.

After obtaining the database, attackers can perform dictionary, brute-force, and precomputed-hash attacks. The problem becomes more serious if the company stores passwords using unsalted MD5 hashes because identical passwords produce identical hash values and attackers can use rainbow tables. Weak and commonly used passwords can therefore be cracked relatively easily.

A more secure approach is to replace MD5 with **PBKDF2 using HMAC-SHA-256 or HMAC-SHA-512**. Each password should be combined with a unique, randomly generated salt, and the hashing process should use a sufficiently high iteration count. The salt and resulting derived value can be stored, but the original password must never be stored.

Using a unique salt prevents attackers from efficiently using precomputed tables, while repeated SHA-based processing makes password guessing more computationally expensive. Therefore, replacing MD5 with a properly configured salted PBKDF2-HMAC-SHA-256/SHA-512 approach provides significantly stronger protection against password-cracking attacks.

2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.

Answer:

Digital signatures are intended to provide authentication, integrity, and non-repudiation for banking transactions. If unauthorized transactions are being approved, there may be weaknesses in the signature-verification process, private-key protection, certificate validation, or transaction design.

One possible flaw is improper verification of the digital signature. The application may accept an invalid signature or may verify the signature against the wrong transaction data. Another serious problem could be a compromised or poorly protected private key. If an attacker obtains a user's private key, they may be able to generate valid signatures. Weak or

outdated cryptographic algorithms and incorrect certificate validation can also reduce security.

The complete transaction information must be included in the signed data. For example, important details such as account number, recipient, amount, and transaction identifier should be cryptographically bound to the signature. Otherwise, an attacker might modify information that was not actually signed.

The bank should use secure standards such as **RSA-PSS, ECDSA, or EdDSA** with appropriate key sizes and trusted cryptographic libraries. Private keys should be securely stored and protected from unauthorized access. The application must validate the signature, public key, certificate chain, certificate validity, and complete transaction data before approving a transaction.

These improvements ensure that the transaction was authorized by the legitimate user and that its contents were not modified. Proper digital-signature verification and strong key management therefore reduce the possibility of unauthorized transactions being approved.

3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.

Answer:

Kerberos is a network authentication protocol that uses tickets, authenticators, timestamps, and session keys to securely authenticate users. A replay attack occurs when an attacker captures valid authentication information from network communication and attempts to reuse it later to gain unauthorized access.

Kerberos normally protects against replay attacks by using timestamps and authenticators. However, the attack may succeed if the server does not correctly validate the timestamp or authenticator, if system clocks are not properly synchronized, or if authentication tickets remain valid for unnecessarily long periods. Incorrect configuration or outdated Kerberos implementations can also weaken replay protection.

For example, an attacker could capture valid authentication information and attempt to send it again. If the server accepts old authentication information without checking its freshness, the attacker could potentially gain access without knowing the user's password.

To prevent this, the enterprise should use properly configured **timestamps, authenticators, session keys, and short-lived tickets**. All client and server systems should maintain accurate and synchronized clocks. Servers should verify ticket lifetimes and ensure that authentication information is fresh and has not already been used. Kerberos servers and related systems should also be regularly updated and securely configured.

These measures make captured authentication information useful for only a very limited period and prevent previously used authentication data from being accepted again. Therefore, correct timestamp validation, ticket management, and authenticator checking provide strong protection against Kerberos replay attacks.

4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.

Answer:

A simple cryptographic hash function generates a fixed-length value, called a hash or message digest, from a message. Although a hash can help detect accidental changes, it does not provide authentication because it does not use a secret key.

The approach fails because an attacker who intercepts a message can modify its contents and calculate a new hash for the modified message. The attacker can then send the modified message together with the new hash. When the receiver calculates the hash, it will match the attacker's hash, so the modification may not be detected.

For example, if the original message is changed from one transaction value to another, the attacker can simply calculate the hash of the changed message. Since calculating a normal hash does not require secret information, the attacker can create a valid-looking hash.

The solution is to use **HMAC-SHA-256** instead of a simple hash. HMAC combines the message with a secret key known only to the authorized communicating parties. The sender generates an HMAC using the message and secret key and sends it with the message. The receiver uses the same secret key to calculate the HMAC and compares it with the received value.

An attacker who does not possess the secret key cannot easily generate a valid HMAC for a modified message. Therefore, HMAC-SHA-256 provides both **message integrity and authentication**, making it much more suitable for protecting messages against active modification.

5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

Answer:

X.509 certificates are used to establish the identity of systems and create trust during secure communication. Frequent trust-validation errors indicate that there may be problems with

certificate validity, certificate configuration, Certificate Authorities, or the certificate trust chain.

One common problem is an **expired certificate**. A certificate may also have been revoked because its private key was compromised or for another security reason. Another issue occurs when the certificate is issued by a Certificate Authority (CA) that is not trusted by the user's system. Missing intermediate CA certificates can also prevent the client from constructing the complete trust chain from the server certificate to the trusted root CA.

Incorrect hostname or domain information is another common cause. The hostname accessed by the user must match the identity information in the certificate. Incorrect system date and time can also cause a certificate to appear expired or not yet valid. Incorrect key-usage settings and poorly managed certificate stores may create additional validation failures.

The organization should obtain certificates from trusted CAs and correctly configure the complete **end-entity** → **intermediate CA** → **trusted root CA** chain. Required intermediate certificates should be installed correctly. Certificate expiry and revocation should be monitored regularly, and certificates should be renewed before they expire. Hostname information must be correctly configured, and all systems should maintain accurate date and time.

These measures ensure that certificates can be properly validated back to a trusted root CA. Regular certificate lifecycle management also prevents expired, revoked, or incorrectly configured certificates from causing authentication failures. Therefore, proper X.509 certificate management and complete trust-chain validation provide reliable authentication and secure communication.